Solving "Choreographer's Block": Transformer-Based Tap Dance Sequence Completion

Amanda Healey
Brandeis University
amandaheal@brandeis.edu

1 Introduction

NLP has been widely applied to domains involving structured symbolic sequences, such as natural language, code, and biological data. In contrast, relatively little work has explored artistic domains, where structure exists but is less commonly formalized for computational modeling. This paper investigates the application of sequence modeling techniques to tap dance choreography, a domain that combines a finite movement vocabulary with well-defined rhythmic and physical constraints.

Tap dance is especially well-suited to sequence modeling. The form consists of a discrete set of steps such as *flap*, *shuffle*, and *ball change*, each associated with specific rhythmic values and constraints on foot placement and weight transfer. Choreography is typically organized into phrases that are commonly eight counts, requiring dancers to satisfy both temporal structure (eighth-note, triplet, and sixteenth-note subdivisions) and physical feasibility (maintaining or transferring weight between feet). These properties make tap sequences structured but non-deterministic, aligning naturally with sequence modeling approaches in NLP.

This project falls under the applied track, with a focus on data representation and construction. I formulate a sequence completion task: given a partial tap sequence annotated with step, foot, and count information, the model has to generate a continuation that is both rhythmically consistent and physically plausible. The central question is whether a model trained on structured choreography data can learn valid step transitions, maintain coherent rhythmic phrasing, and respect constraints such as weight transfer.

Beyond its technical framing, this task is motivated by a practical application. In choreography, it is common to construct a partial sequence and then struggle to extend it in a musically and physically coherent way. This is a dance equivalent of *writer's block*. I explore whether a language model can help in this process by generating plausible continuations that adhere to the underlying structure of the form. This work aims to bridge NLP and dance by demonstrating how sequence modeling can be applied to a structured artistic domain.

2 Related Work

Research on computational representations of dance has primarily focused on formal notation systems and motion synthesis. The most established notation framework is known as *Labanotation*, a symbolic system for encoding human movement that represents spatial, temporal, and dynamic aspects of motion (Hutchinson Guest, 1984; Laban, 1920). While expressive and widely used in dance documentation, *Labanotation* is designed for human interpretation and archival purposes rather than any sort of machine learning or sequence prediction.

More recent work has focused on dance motion synthesis using deep learning models that have been trained on motion-capture data. These approaches represent dance as continuous 3D pose sequences and generate motion, conditioned on music or learned motion priors. For example, music-conditioned dance generation systems learn mappings between audio features and motion trajectories using neural sequence models (Lee et al., 2019). These methods are effective for realistic motion generation, but they differ from this project in the sense that they operate in continuous pose space rather than discrete symbolic step representations.

Tap dance also has existing practitioner-developed symbolic notation systems, most notably *Kahnotation*, which encodes choreography using discrete symbols for steps, rhythmic placement, and foot usage. These systems function as a form

of shorthand transcription for dancers, allowing sequences to be recorded in a linear symbolic format. However, unlike more formal movement notation systems such as *Labanotation*, *Kahnotation* is not standardized within computational or academic contexts and is primarily used pedagogically within small tap communities. As a result, it has not been widely adopted in machine learning settings and does not directly support sequence modeling or generative tasks. This highlights a gap between human-readable dance transcription systems and representations suitable for data-driven modeling of choreography.

Overall, existing work either focuses on continuous motion generation or symbolic documentation, while relatively little research addresses discrete, rhythmically structured dance as a sequence modeling problem. This project bridges that gap by framing tap dance choreography as a symbolic sequence completion task, where models must learn valid step transitions, rhythmic structure, and constraints such as weight transfer.

3 Data

3.1 Dataset Source and Composition

The dataset consists of my personal tap dance class plans collected over a course of several years of teaching. Of course, class plans were filtered purely to the tap genre, as the step vocabulary is quite distinct from other genres. The difficulty level of the exercises ranges from around age 10 up through the advanced college level. Each document corresponds to a single class session and contains multiple choreographic exercises embedded within structured lesson notes.

In total, my raw corpus contains 31 class plan documents, each including approximately 5–10 exercises per document. However, not all exercises were suitable for model training. After filtering for consistency in rhythmic structure, clarity of notation, and pedagogical redundancy (for instance, repeated content across weeks or exercises with incompatible time signatures), the final dataset contains 106 annotated exercises.

While this is quite small compared to standard NLP corpora, the dataset is highly structured and densely annotated within a specialized domain. The sheer amount of information encoded in these exercises makes it a strong fit for automated data augmentation, eventually resulting in a strong training corpus.

Raw Class Notes
9:13-9:18: 2 LINES DIFF DIRECTIONS: Send Me On My Way • Do this briefly, we started it last week • 2 lines. Go through windows (one line does this pass coming fwd) • (***review backward pass for the other line) • FORWARD PASS: • Flap ball change, flap ball change • quick sh ball HEEL x3 (sixteenth; accent is not on the shuffles), ball change • Paradiddle L, stamp stamp stamp (3&4), paradiddle, stamp, stamp (7 8) • Pause between, THEN repeat new music

Table 1: Raw instructor-generated tap dance class notes.

3.2 Raw Data Characteristics

As shown in Table 1, the raw data alone would not be a useful training signal for a model. The data consists of unstructured notes combining abbreviated step names (e.g., “sh” for shuffle), informal capitalization for emphasis, partial rhythmic annotations on some steps but certainly not all (e.g., “3&4”, “7 8”), and implicit information such as foot usage and weight transfer. This is why the task required manual annotation as the first step.

This format is highly interpretable to humans (albeit lacking in rhythmic specificity) but not directly suitable for sequence modeling due to inconsistent naming, implicit structure, and missing explicit annotations.

3.3 Preprocessing and Standardization

A multi-stage preprocessing pipeline was used to convert raw notes into structured symbolic sequences. A custom annotation and preprocessing script was developed to operate in a semi-interactive human-in-the-loop manner, where each class document is processed line-by-line. The annotator incrementally constructs each exercise by explicitly specifying step, count, and foot information.

For each completed exercise, the script serializes the resulting structured sequence into a normalized representation and writes it to a JSONL file for downstream model training. This process en-

sures consistency in formatting while still allowing expert-level interpretation of ambiguous or underspecified instructional notes.

The annotation interface also supports interactive correction by allowing the user to undo steps during construction, along with controlled exercise termination, enabling detailed curation of training examples directly during preprocessing.

In addition to encoding the steps during this preprocessing phase, each sequence is annotated with three other fields: song name, "repeat" (taking either "same", "reverse", or "none" to indicate if the exercise repeats on the same foot or not when repeated twice consecutively), and "travel" (taking a boolean value to indicate whether or not the exercise travels across the floor or is performed stationary in the center of the room). These fields remain in the final processed data for the potential of inclusion in downstream tasks later on, though for the purpose of this sequence completion project they remain unused.

3.3.1 Step Name Standardization

All step names were normalized to lowercase and expanded into canonical forms (like “sh” expanding to "shuffle"). Inconsistent terminology was resolved, as well. For example, the original documents used the terms "pickup" and "pullback" interchangeably, so I standardized this by unifying them all under the label "pullback".

3.3.2 Sequence Representation

Each exercise was converted into a sequence of atomic tokens of the form (*step*, *foot*, *count*), where "step" denotes the movement type. The information encoded in "foot" indicates execution side: L, R, B (both), N (neither, reserved for movements like "clap" that do not involve footwork but still play an important role rhythmically), or combinations of these. As an example of a combination, a *ball change* is encoded with two feet (LR or RL), because it arguably places equal emphasis on each foot, and a *pas de bourree* is encoded with three feet (LRL or RLR) for the same reason. The "count" field represents rhythmic placement. Counts never exceed 8, and they may be broken down into any of the following rhythms, or any combination of them:

- Quarter notes: 1 2 3 4
- Eighth notes: 1& 2& 3& 4&
- Triplets: 1&a 2&a 3&a 4&a
- Sixteenth notes: 1e&a 2e&a 3e&a 4e&a

Counts also allow for pauses. That is, one step may occur on count 7, and the next step may occur on count 1 on the following phrase, indicating a pause on count 8.

Crucially, the number of characters in this count field directly corresponds to the number of sounds in the step. For example, a paradiddle happening on "5e&a" contains exactly 4 sounds.

3.3.3 Atomic vs. Composite Steps

Some composite steps such as *flap heel* were retained as step names, instead of breaking them down into their components (*flap* followed by *heel drop*) due to their frequency in the data and their interpretability. In contrast, rarer or more complex steps (e.g., *Maxie Ford*) were decomposed into simpler components (*step, shuffle, step, toe*) to improve generalization and reduce sparsity.

3.3.4 Rhythmic and Structural Normalization

Several simplifying assumptions were introduced:

- Cramp rolls were reduced to their initiating foot. Even though a standard cramp roll makes 4 sounds, the step is defined by its initiation. The count field maintains 4 characters to account for the sounds, but only one *foot* is encoded. This differs from the earlier discussion of ball changes and pas de bourrees.
- Step distinctions such as *heel* (hitting the back edge of the heel tap on the floor) vs. *heel drop* (starting with weight already placed on the working foot, then just lifting the heel tap and hitting it back into the floor) were preserved due to functional differences.
- A large range of terminology is used to indicate placing a foot down and shifting weight onto it, such as *stamp*, *step*, *jump*, *leap*, and *drag*. These were all standardized to become *step* to make them easier for the model to recognize, with the exception of *stamp* (because stamps are differentiated by the fact that they involve a heavier accent, using the entire foot).

All sequences were normalized to fit an 8-count structure. This involved minimal changes, as most already fit this constraint. Irregular phrases, or those that occurred in a 3/4 or 6/8 time signature rather than one like 2/4 or 4/4 that lends itself to 8 counts, were either manually completed to fill the

Processed Class Sequence


```
{
"song": "Send Me On My Way",
"sequence":
"flap|R|&1 ball_change|LR|&2
flap|L|&3 ball_change|RL|&4
shuffle|R|e& ball_heel|R|a5
shuffle|L|e& ball_heel|L|a6
shuffle|R|e& ball_heel|R|a7
ball_change|LR|&8
paradiddle|L|1&2&
stamp|R|3 stamp|L|& stamp|R|4
paradiddle|L|5&6&
stamp|R|7 stamp|L|8",
"repeat": "same",
"travel": true
}
```


Table 2: Example of a cleaned structured tap dance training instance.

phrase, or excluded if temporal reconstruction was unreliable.

The sequence from Table 1 fully processed can be seen in Table 2.

3.4 Data Augmentation

Given the relatively small dataset size (106 exercises), augmentation strategies are used to improve generalization and reduce overfitting to specific step sequences, footings, and rhythmic patterns. The augmentation pipeline operates directly on a structured symbolic representation of tap dance sequences, where each token encodes a step type, foot assignment, and rhythmic count string. The dataset before augmentation effectively serves as a baseline; the model itself is underlyingly the same on both this baseline and the full dataset, as the intent was to determine how much of a difference dataset scale would make.

Three primary augmentation mechanisms are applied: temporal shifting, spatial mirroring, and structural transformation.

Temporal shifting is implemented by cyclically rotating rhythmic count positions within an 8-unit grid, preserving internal subdivisions such as “&”, “e”, and “a”, while shifting beat indices forward by a fixed offset of 4. This increases robustness to phase variation within measures without altering the underlying step structure. Because the task is sequence continuation, the model is trained on prefixes of step sequences and only needs to learn valid local transitions rather than fixed associations between specific steps and absolute counts.

Spatial mirroring is applied by swapping left and right foot assignments throughout the sequence while preserving step identity and rhythmic alignment. This encourages the model to learn foot-invariant representations of movement patterns.

Structural transformation consists of step-level augmentation, including probabilistic step substitutions (swaps) and hierarchical expansions of compound steps into shorter atomic constituents. During expansion, a single high-level step like a riff, which is composed of several sounds, may be decomposed into multiple steps according to a manually-predefined grammar. Foot assignment during these expansions is governed by explicit relational rules: same-foot propagation, opposite-foot mapping, or mixed patterns such as opposite-then-same. This ensures that multi-step decompositions remain rhythmically consistent and still physically possible. One example of this is the expansion of a pas de bourree encoded with the feet "RLR" on counts "1&2" into a sequence of a shuffle on R (counts 1&), followed by a step on R (count 2).

Since there is a degree of randomness to the structural transformations, duplicated sequences are possible; these duplicate augmented sequences are filtered using a deduplication set to prevent redundancy. This combination of temporal, spatial, and structural augmentation increases dataset diversity while preserving the underlying rhythmic and biomechanical constraints of tap dance vocabulary. The final augmented dataset contains 628 examples, which is approximately 6 times the size of the original set. 10% of the data was held out for validation. One flaw to this process is the possibility of partial data on the validation set. There are no full duplicate sequences; however, the augmentation process does not guarantee entirely novel exercises or rhythms. This is in part acceptable due to the fact that tap dance inherently includes a great deal of repetition, but nonetheless qualitative testing on unseen data was performed later on to address this issue.

The final dataset numbers are as follows:

- **Baseline**: 106 total examples (no augmentation); training set of 91, validation set of 15
- **With augmentation**: 628 total examples; training set of 566, validation set of 62

4 Methods

4.1 Training Objective

The model is trained using a standard next-token prediction objective over structured tap dance sequences. Given a sequence of tokens encoding step, foot, and count, the model learns to predict each subsequent token conditioned on all previous tokens.

Unlike typical instruction-tuning setups, no explicit input/output split is used. Each exercise is treated as a single continuous sequence during training. This design choice avoids introducing artificial segmentation boundaries and allows the model to learn both local transition dynamics step-to-step and also higher-level structure such as phrasing and rhythmic completion across full sequences.

The default Alpaca-style instruction formatting used in the base training notebook was therefore removed. Since the task does not involve unique instructions beyond "finish this structured dance sequence", enforcing an instruction-input-output schema would introduce unnecessary bias toward arbitrary split points within sequences.

4.2 Model and Training Framework

Training was implemented using the standard supervised fine-tuning (SFT) pipeline provided in the Unsloth notebook, with modifications to accommodate the custom sequence format, adjust hyperparameters, and compute custom evaluation metrics (Han and Han, 2023). Parameter-efficient fine-tuning with LoRA enabled training of the Llama 3.1 8B model within the memory constraints of a single T4 GPU in Google Colab.

4.3 Training Setup

As an initial baseline, the model was trained directly on the 91 exercises in the training set without any data augmentation. Each exercise was serialized into a single text sequence and used as an individual training example.

Training used the Hugging Face TRL library's `SFTTrainer` framework. On this baseline, the model was trained for 6 epochs, as shorter training runs, such as just a single epoch, resulted in unstable loss behavior and clear under-training. A per-device batch size of 2 was used with gradient accumulation over 4 steps (effective batch size of 8), following the configuration provided by the training framework.

The learning rate was reduced from the default setting to 1×10^{-4} to stabilize training on the small dataset, and weight decay was increased to 0.01 to introduce additional regularization and attempt to reduce overfitting on so few examples. Optimization was performed using the 8-bit AdamW optimizer with a linear learning rate schedule and a short warmup phase, following the default training configuration.

Sequence packing was disabled (`packing = False`) so that each training example corresponded to a complete exercise rather than concatenated sequences. This ensures that the model learns coherent full-phrase structure instead of artificial transitions between unrelated exercises. The maximum sequence length was set to accommodate full exercise representations.

When the model training was extended to the full augmented dataset, epochs were reduced to 3, as there were many more examples that could provide model signal, so 6 total epochs would only result in overfitting. Other hyperparameters, including learning rate and weight decay, were kept the same as the baseline training run.

5 Results

5.1 Evaluation Setup

To evaluate the baseline and full datasets, I performed an automated sequence completion test on both the baseline and augmented validation sets. From these sets, a random prefix of each sequence was selected and provided to the model. The model was then tasked with generating the remainder of the sequence. The complete generation was passed through a rule-based evaluator that checks structural validity, rhythmic consistency, vocabulary correctness, and basic physical plausibility of movement transitions.

LLM Disclosure: Gemini Pro was used as a coding assistant to help implement the rule-based evaluation logic, specifically for translating my custom rules for weight transfers and error-type categorization into Python. All underlying domain logic and tap dance constraints were designed by hand, and the resulting code for this step in the pipeline was fully reviewed.

Due to the nature of the data augmentation process, augmentations were limited in how much variation they provided, beyond simple swaps and step expansions based on a fixed mapping. This means that training and validation sets could potentially

have very similar sequences. In order to offset the chance of partial data leakage here, a second testing approach was taken on the model after training on the full dataset, as described in Subsection 5.5: generating completely unseen sequence prefixes to provide to the model, then qualitatively inspecting the model generations.

5.2 Evaluation Criteria

Each completed sequence based on the validation set was classified into one of five categories:

- PASSED: All of the structural, rhythmic, vocabulary, and physics constraints are satisfied.
- SYNTAX ERROR: The sequence exhibits invalid formatting or is missing required structure (step|foot|count).
- VOCAB ERROR: The output includes an unknown step token.
- PHYSICS ERROR: The sequence violates constraints on foot state transitions, including weight-bearing requirements defined in the step physics model.
- RHYTHM ERROR: The temporal ordering of counts is invalid.

An additional evaluation metric penalized for individual invalid steps but allowed for recovery later in the sequence. This involved tracking which foot weight is placed at the beginnings and ends of each step and determining if transitions are physically possible, then turning this number of valid transitions into a percentage. This evaluation metric resets the state after an error in order to allow for partial credit and provides a more informative exploration of model output.

5.3 Baseline Results

6 total epochs on the baseline run allowed the loss to stabilize without causing it to plateau and then increase on the validation set. Validation loss steadily decreased, flattening out at 0.5762 by step 70 without increasing. This indicates optimal generalization before overfitting. The training and validation loss values at 10-step intervals are detailed in Table 3, and the overall trajectory is visualized in Figure 1.

On the 15 examples that were held out for validation after baseline training, the model produced the following distribution of error types under the strict, break-on-failure evaluation logic:

Table 3: Training and Validation Loss from Baseline Dataset (70 Steps)

Step	Training Loss	Validation Loss
10	1.4187	1.1744
20	0.6851	0.7826
30	0.6465	0.6745
40	0.5982	0.6291
50	0.3717	0.5914
60	0.5755	0.5808
70	0.3516	0.5762

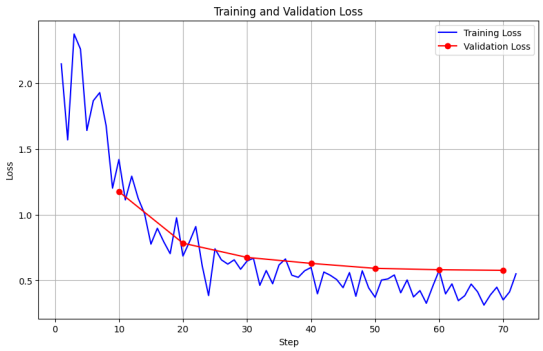


Figure 1: Loss Curve for Baseline Model showing convergence at 70 steps.

- PASSED: 4 (26.7%)
- SYNTAX ERROR: 1 (6.7%)
- VOCAB ERROR: 0 (0.0%)
- PHYSICS ERROR: 9 (60.0%)
- RHYTHM ERROR: 1 (6.7%)

In the partial credit metric evaluation, which counts each step transition independently, the model achieved a validity percentage of 66.8%.

5.4 Final Model Results

The full dataset, after augmentation, took 3 epochs to converge. The training and validation loss were tracked in steps of size 30, seen in Table 4. The loss curve is shown in Figure 2; loss on this full dataset converged to a lower value than the smaller dataset.

On the augmented validation set, the initial evaluation logic produces the following errors:

- PASSED: 18 (29.0%)

Table 4: Training and Validation Loss from Full Augmented Dataset (210 Steps)

Step	Training Loss	Validation Loss
30	0.6662	0.6645
60	0.4735	0.5213
90	0.4468	0.4676
120	0.4668	0.4257
150	0.3361	0.4129
180	0.2747	0.3725
210	0.3064	0.3624

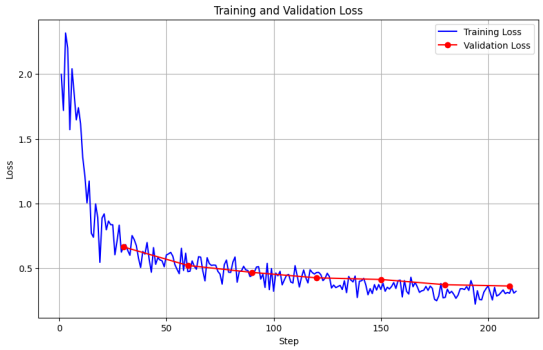


Figure 2: Loss Curve for Final Model on Augmented Data.

- SYNTAX ERROR: 2 (3.2%)
- VOCAB ERROR: 0 (0.0%)
- PHYSICS ERROR: 40 (64.5%)
- RHYTHM ERROR: 2 (3.2%)

This logic above essentially performs a binary classification of each sequence as valid or invalid, not allowing for degrees of correctness to be represented. The "passed" percentage of 29.0% does not seem to substantially outperform the baseline of 26.7%, but a valid step percentage of 75.9% with the more nuanced partial credit metric suggests otherwise.

5.5 Behavior on Completely Unseen Data

As discussed above, the potential for some leakage of sequences from training into validation sets calls for a metric on completely unseen data. To do this, I hand-crafted a test set: I provided brand new (but still completely structurally valid) prefixes of varying lengths, similarly to how the evaluation on the validation sets was performed. These were generated manually to both ensure logic and also simulate the way that this model is intended to be deployed for real use. I inspected outputs for a total of 20 generations, after passing them to the model that had been trained on the augmented training dataset of 566 sequences. Among these 20 generations, average per-step (partial credit allowed) validity came to 74.02%. 13 of the 20 sequences (65.0%) were completely physically plausible with no errors.

6 Analysis & Discussion

The model's behavior can be viewed from both a quantitative and qualitative lens, though qualitative inspection may prove more useful. A key challenge in this task is that the optimization objective (cross-entropy loss over token sequences) is only loosely aligned with the actual notion of correctness, creativity, and naturalness in tap choreography generation. This task is not true natural language modeling, because the model is trained on a structured, domain-specific sequence language. Although a pretrained LLaMA-based model is used due to its strong compositional generalization capabilities, this task differs significantly from standard NLP in many ways: tokens are now encoding procedural actions rather than semantics, validity depends on physical constraints like weight transfer and timing, and the sequence structure behaves more like a formal grammar than a free-form language.

In both the binary classification of sequence outputs as valid or invalid, and in the individual step validity checker metric, the full dataset after augmentation led to better results than the baseline set. That said, limitations of this evaluation include the extremely small validation set in the baseline (n = 15), and the potential for some data leakage in the augmented file. This makes quantitative metrics somewhat unstable when considering the baseline.

Though an LLM-as-a-judge approach was considered, this was not pursued due to a circular problem: using a language model to evaluate another language model on structured sequence validity risks passing along the same biases and failure modes. In this setting, rule-based evaluation is generally more reliable because identifying whether a sequence is valid is not entirely subjective. Despite the open-ended nature of sequence generation, validity can be defined using explicit constraints: syntactic validity of `step|foot|count` structure, foot legality constraints, temporal/count progres-

sion consistency, and expansion correctness under deterministic rules. Therefore, while the output space is large, it is not ambiguous in a linguistic sense; it is a structured symbolic system. This makes rule-based evaluation preferable because it is deterministic and reproducible, and it directly encodes domain constraints in a mechanistically-interpretable way.

6.1 Qualitative Error Analysis

Beyond metrics alone, the evaluation of specific error types highlights several recurring failure modes.

Physics errors made up the vast majority of model failures. The model frequently assigned movements to a non-weight-bearing foot, or violated foot transition constraints. For instance, consider the following result, generated after training on the full dataset:

- Prefix: `heel_toe|R|1&` `heel_toe|L|2&` `heel_toe|R|3&` `heel_toe|L|4&` `ball_heel|R|5&` `heel_toe|L|6&` `ball_heel|R|7&` **`clunk|L|8`**
- Generated: **`heel_toe|R|1&`** `ball_heel|L|2&` `heel_toe|R|3&` `ball_heel|L|4&` `heel_toe|R|5&` `heel_toe|L|6&` `ball_heel|R|7&` `clunk|L|8`

Though the counts are accurate and the exercise is otherwise physically plausible, the transition from `clunk|L|8` to `heel_toe|R|1&` is physically impossible, because the foot that performs the clunk does not bear any weight. So, the weight is already in the right foot after the clunk, and there is no way to pick it up and perform a heel-toe on the right foot without some intermediate step.

There were rare instances where the model produced malformed tokens, leading to syntax errors, and rare cases of rhythm errors where the model did not successfully count a phrase or mistakenly counted up to 9 when it was only trained on 8-count phrases.

6.2 Unseen Data Performance

Perhaps the most useful strategy in evaluation was the process of validating model outputs on unseen data by hand. The quantitative metrics discussed above provide useful information on this unseen data performance. The full success of 65% of these sequences is a great improvement but likely due to noise; this may be attributed to relatively simpler prefixes or the mere lack of stability of a set of just 20 generations. It is worth noting, though, that "physically possible" is not interchangeable with "good" and this is surely a limitation of the evaluation strategy rather than the model's ability to learn. For instance, the eighth generated sequence among these 20 included the transition `"toe|L|& spank|L|4"`. In terms of weight transfer, this is perfectly acceptable, though the angle of the foot on the toe motion makes a spank awkward and unnatural. With the second metric, the per-step validity (74.02%) is quite close to the augmented validation set (75.9%), suggesting that leakage did not have a significant effect.

Aside from the evaluation metrics of the 20 generations from unseen prefixes, a reassuring takeaway is the fact that errors *outside* of pure weight transfer almost entirely involved less common steps. Flaps, shuffles, and steps appear everywhere, but when I introduce a prefix including wings or riffs, for instance, the model has had less exposure and is consequently less familiar with how to complete the phrase. The mere fact that the model even attempts to include riffs in the completion of a phrase that has begun with riffs is promising behavior, because it shows that the model is capable of producing some abstract sense of cohesion, so that the beginning and end of the exercises feel connected.

Remarkably, among the errors in this process, almost every single one involved a single *character* within the erroneous part of the token. Examples of this include the following:

- Encoding a flap heel as 2 sounds, on the counts 8&, when in reality this step should always be 3 sounds, like &8&.
- Including a 4-count riff instead of a 3-count riff. Simply changing this 4 to a 3 in one particular example would have made the sequence's counts and weight transfers align perfectly.
- Performing a single switch wing on the right instead of the left foot, which would have kept the dancer on the proper foot for the next step.

One final note on these novel sequence completions is their variation from the training data. Upon examination, it was clear that none of the outputs were an exact match to a single training instance, supporting the idea that the model is not merely

copying unoriginal patterns. This is one element of model performance that can be pushed further, perhaps by raising temperature to allow for even more creative outputs.

6.3 Limitations of the Dataset and Evaluation Framework

The dataset has several structural limitations. For one, it does not explicitly encode spatial orientation, rotation, or traveling information. The representation used in this exploration, therefore, is plausible for tap since it is a footwork-heavy genre, but for other styles of dance, a richer movement encoding would be necessary. The rigid count structure also prevents any expression of syncopation, which is a fairly frequent and often musically-driven component. Finally, tap dance terminology is not universal, and other instructors may not find use in a system that was programmed entirely based off of a single dancer's training and knowledge, limiting the potential for it to be deployed on a larger scale. These factors constrain both model performance and evaluation fidelity, and should be considered when interpreting results.

When inspecting the errors identified by the evaluation pipeline, it became apparent that not every error was a *true* error. This is because of ambiguous details in some footwork, such as the fact that a "step" necessarily shifts weight onto the working foot, but this does not imply that the other foot must be completely off the ground, which is what the model assumed. This problem comes from the way that the weight transfers were encoded, and the meanings of "free" (no weight is placed in the foot) and "bearing" (weight is anchoring the foot to the ground) when applying the rules. The physics evaluation, then, is perhaps too rigid to reflect real life variability in movement. In future work, this nuance could be captured in a more fine-grained way, particularly if the data included a larger range of steps exhibiting this weight state. This also further stresses the importance of human-in-the-loop validation to inspect a subset of the generated sequence, in search of misconceptions such as this one. Ultimately, the flaws in physics evaluations were relatively rare and do not entirely undermine the reliability of the rule-based evaluation process.

6.4 Summary of Observations

The model successfully generates structurally valid continuations in a small subset of cases, but the majority of failures occur due to violations of the learned physical constraints governing weight transfer between feet. Syntax and vocabulary errors are relatively rare, suggesting that the model learns the token format and step vocabulary more reliably than higher-level structural constraints.

These results establish a promising first attempt at structured sequence continuation in the tap dance domain, both with and without data augmentation. The primary limitation on model performance appears to be dataset size and lack of variation in the data.

7 Conclusion

This exploration demonstrates that a pretrained transformer model can be adapted to generate structured tap dance sequences. The model shows clear potential to learn local rhythms and foot transition behavior, particularly when supported by rule-based constraints and data augmentation. However, the results also reinforce a key limitation: minimizing training loss does not guarantee the production of valid sequences under domain-specific constraints.

Several directions for future work emerge from this project. First, scaling the dataset is likely the most meaningful improvement. Because this representation is manually constructed, continued annotation of exercises during class planning provides a natural path to incrementally expand the dataset. With substantially more data, the model could better capture a more diverse range of plausible structures and reduce failure rates in rhythm and constraint tracking.

Second, the representation itself can be extended. Currently, each token encodes `step|foot|count`, but additional dimensions such as body orientation or direction of travel could be incorporated as additional fields. This would allow the model to learn richer choreographic structure beyond purely rhythmic and foot-based constraints. These sorts of fields would open up more possibility for the framework to extend to other dance styles. However, this is challenging because unlike tap, many styles have less constrained vocabularies and more continuous movement spaces. Adapting this approach would require redefining the tokenization scheme and constructing a new annotated dataset. In practice, this would require a significantly larger and more diverse dataset than was feasible for this project.

In terms of developing a more comprehensive

evaluation process, it should be mentioned that there is more to a tap sequence that makes it satisfactory than just physical and rhythmic constraints. Dance as an art form values creativity, which suggests that an ideal model would produce subjectively interesting and unique combinations of steps. Furthermore, even those sequences that are physically possible in a technical sense might not be the most natural to execute, as described in Section 6.2. These qualities of creativity and naturalness are distinct from the validity evaluated by this project's rule systems, so a metric to address them would be useful. In this case, using a powerful LLM that has been trained on high-quality sequences could offer an additional qualitative layer of insight.

Finally, there is potential for downstream applications such as choreography assistance or music alignment. For example, sequences could be paired with tempo, energy level, and other non-textual musical data to generate rhythmically compatible choreography. However, this would require careful handling of timing modifications and alignment between symbolic counts and audio features.

Overall, this project shows that transformer models can learn meaningful structure in a highly constrained, domain-specific sequence language, but also underscores the importance of representation design, evaluation methodology, and dataset scale in determining success.

References

Daniel Han and Michael Han. 2023. Unsloth: Make llm finetuning 2x faster and 70% less memory. https://github.com/unslothai/unsloth.

Ann Hutchinson Guest. 1984. *Labanotation: The System of Analyzing and Recording Movement*. Routledge.

Rudolf Laban. 1920. Foundations of movement analysis (laban movement theory).

Juheon Lee, Seohyun Kim, and Kyogu Lee. 2019. Listen to dance: Music-driven choreography generation using autoencoders. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.